

```

"use client";

import React, {
  createContext,
  useContext,
  useMemo,
  useRef,
  useEffect,
  useState,
  useCallback,
} from "react";
import { Circle } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export interface RadioOptionItemConfig {
  value: string;
  label: string;
  disabled?: boolean;
}

export interface RadioGroupProps {
  name?: string;
  value: string;
  onChange: (value: string) => void;
  options: RadioOptionItemConfig[];
  allowClear?: boolean;
  disabled?: boolean;
  error?: string;
  orientation?: "horizontal" | "vertical";
  className?: string;
}

interface RadioGroupContextValue {
  name: string | undefined;
  value: string;
  onChange: (value: string) => void;
  allowClear: boolean;
  disabled: boolean;
  error: string | undefined;
  orientation: "horizontal" | "vertical";
  registerOption: (
    value: string,
    ref: React.RefObject<HTMLInputElement | null>,
  ) => void;
}

```

```

    ) => void;
    unregisterOption: (value: string) => void;
    focusedValue: string | null;
    setFocusedValue: (value: string | null) => void;
  }

  const RadioGroupContext = createContext<RadioGroupContextValue | null>(null);

  function useRadioGroupContext() {
    const context = useContext(RadioGroupContext);
    if (!context) {
      throw new Error(
        "RadioGroup compound components must be used within RadioGroup",
      );
    }
    return context;
  }

  interface RadioOptionProps {
    option: RadioOptionItemConfig;
    className?: string;
  }

  const RadioOption = React.forwardRef<HTMLInputElement, RadioOptionProps>(
    ({ option, className }, ref) => {
      const {
        name,
        value,
        onChange,
        allowClear,
        disabled,
        error,
        registerOption,
        unregisterOption,
        setFocusedValue,
      } = useRadioGroupContext();

      const inputRef = useRef<HTMLInputElement>(null);
      const isChecked = value === option.value;
      const isDisabled = disabled || option.disabled;

      useEffect(() => {
        registerOption(option.value, inputRef);
        return () => unregisterOption(option.value);
      }, [option.value, registerOption, unregisterOption]);

      useEffect(() => {
        if (inputRef.current) {
          inputRef.current.checked = isChecked;
        }
      })
    }
  );

```

```

    }, [isChecked]));

const handleChange = useCallback(() => {
    if (isDisabled) return;

    if (allowClear && isChecked) {
        onChange("");
        return;
    }

    onChange(option.value);
}, [isDisabled, allowClear, isChecked, onChange, option.value]);

const handleKeyDown = useCallback(
    (event: React.KeyboardEvent<HTMLInputElement>) => {
        if (isDisabled) return;

        const options = Array.from(
            document.querySelectorAll<HTMLInputElement>(
                `input[name="${name}"][type="radio"]:not(:disabled)`
            ),
        );

        if (options.length === 0) return;

        const currentIndex = options.findIndex(
            (el) => el === event.currentTarget,
        );
        if (currentIndex === -1) return;

        let nextIndex = currentIndex;
        let shouldNavigate = false;

        if (event.key === "ArrowUp" || event.key === "ArrowLeft") {
            event.preventDefault();
            nextIndex = (currentIndex - 1 + options.length) % options.length;
            shouldNavigate = true;
        } else if (event.key === "ArrowDown" || event.key === "ArrowRight") {
            event.preventDefault();
            nextIndex = (currentIndex + 1) % options.length;
            shouldNavigate = true;
        } else if (event.key === "Home") {
            event.preventDefault();
            nextIndex = 0;
            shouldNavigate = true;
        } else if (event.key === "End") {
            event.preventDefault();
            nextIndex = options.length - 1;
            shouldNavigate = true;
        }
    }
);

```

```

        if (shouldNavigate) {
          const nextOption = options[nextIndex];
          if (nextOption) {
            nextOption.focus();
            nextOption.click();
          }
        }
      },
      [isDisabled, name],
    );

    const handleFocus = useCallback(() => {
      setFocusedValue(option.value);
    }, [option.value, setFocusedValue]);

    const handleBlur = useCallback(() => {
      setFocusedValue(null);
    }, [setFocusedValue]);

    const radioClasses = cn(
      "relative inline-flex items-center justify-center",
      "w-5 h-5 shrink-0",
      "rounded-full border-2",
      "border-border bg-background",
      "transition-all duration-200 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
      "focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50",
      "disabled:cursor-not-allowed",
      "aria-invalid:border-destructive aria-invalid:ring-3",
      "aria-invalid:ring-destructive/20",
      isChecked
        ? "bg-brand-primary border-brand-primary text-white shadow-sm"
        : "border-brand-primary/50 border-brand-primary/5",
      error && "border-destructive focus-visible:ring-destructive",
      className,
    );

    const innerDotClasses = cn(
      "transition-all duration-200 ease-out",
      isChecked ? "scale-100 opacity-100" : "scale-0 opacity-0",
    );

    const labelClasses = cn(
      "text-sm font-medium leading-none",
      "transition-colors duration-200 ease-out",
      "text-text-main",
    );

```

```

        error && "text-destructive/90",
        isDisabled && "opacity-50",
        "ml-2",
    );

    const containerClasses = cn(
        "inline-flex items-center cursor-pointer select-none",
        isDisabled && "opacity-50 pointer-events-none cursor-not-allowed",
    );

    return (
        <label className={containerClasses}>
            <input
                ref={(el) => {
                    inputRef.current = el;
                    if (ref) {
                        if (typeof ref === "function") {
                            ref(el);
                        } else {
                            ref.current = el;
                        }
                    }
                }}
                type="radio"
                name={name}
                value={option.value}
                checked={isChecked}
                disabled={isDisabled}
                onChange={handleChange}
                onKeyDown={handleKeyDown}
                onFocus={handleFocus}
                onBlur={handleBlur}
                aria-checked={isChecked}
                aria-invalid={error ? "true" : "false"}
                aria-disabled={isDisabled}
                aria-required={true}
                className="sr-only peer"
            />
            <div className={radioClasses} aria-hidden="true">
                <Circle
                    className={cn("w-2.5 h-2.5", innerDotClasses)}
                    aria-hidden="true"
                />
            </div>
            <span className={labelClasses}>{option.label}</span>
        </label>
    );
},
);
RadioOption.displayName = "RadioOption";

```

```

export function RadioGroup({
  name,
  value,
  onChange,
  options,
  allowClear = false,
  disabled = false,
  error,
  orientation = "vertical",
  className,
}: RadioGroupProps) {
  const [focusedValue, setFocusedValue] = useState<string | null>(null);
  const optionRefs = useRef<
    Map<string, React.RefObject<HTMLInputElement | null>>
  >(new Map());

  const registerOption = useCallback(
    (optionValue: string, ref: React.RefObject<HTMLInputElement | null>) => {
      optionRefs.current.set(optionValue, ref);
    },
    [],
  );

  const unregisterOption = useCallback((optionValue: string) => {
    optionRefs.current.delete(optionValue);
  }, []);

  const contextValue = useMemo<RadioGroupContextValue>(
    () => ({
      name,
      value,
      onChange,
      allowClear,
      disabled,
      error,
      orientation,
      registerOption,
      unregisterOption,
      focusedValue,
      setFocusedValue,
    }),
    [
      name,
      value,
      onChange,
      allowClear,
      disabled,
      error,
      orientation,
    ]
  );

```

```

        registerOption,
        unregisterOption,
        focusedValue,
        setFocusedValue,
    ],
);

const groupClasses = cn(
    "inline-flex",
    orientation === "horizontal" ? "flex-row gap-6" : "flex-col gap-2",
    className,
);

return (
    <RadioGroupContext.Provider value={contextValue}>
        <div
            className={groupClasses}
            role="radiogroup"
            aria-label={name}
            aria-invalid={!error}
            aria-disabled={disabled}
            aria-orientation={orientation}
        >
            {options.map((option) => (
                <RadioOption key={option.value} option={option} />
            ))}
            {error && (
                <p
                    className={cn(
                        "text-sm",
                        "text-destructive/90",
                        "transition-colors duration-200 ease-out",
                        orientation === "horizontal" ? "ml-10 mt-1" : "mt-1 ml-7",
                    )}
                    role="alert"
                    aria-live="polite"
                >
                    {error}
                </p>
            )}
        </div>
    </RadioGroupContext.Provider>
);
}
RadioGroup.displayName = "RadioGroup";

export { RadioGroupContext, RadioOption };

```